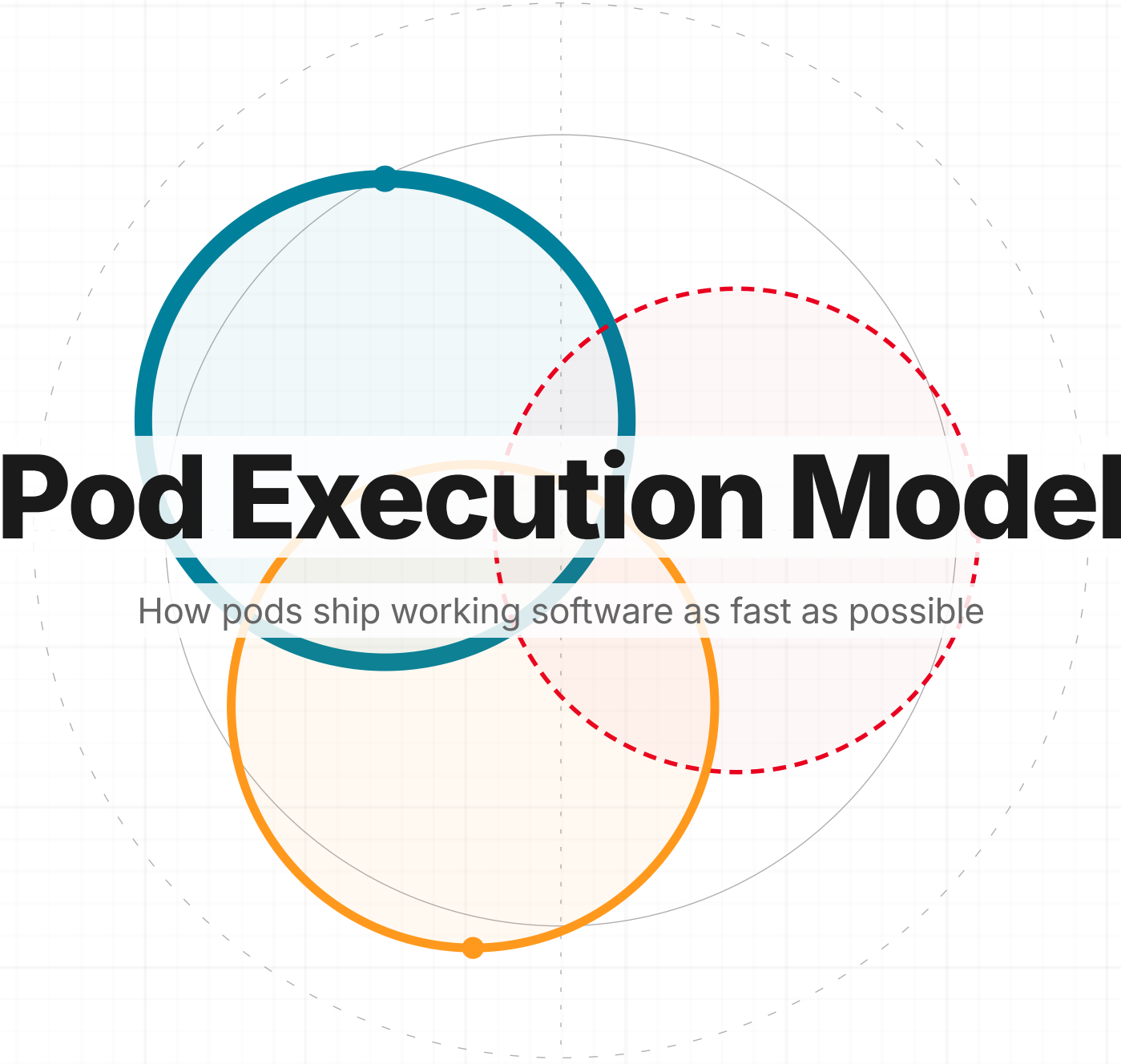
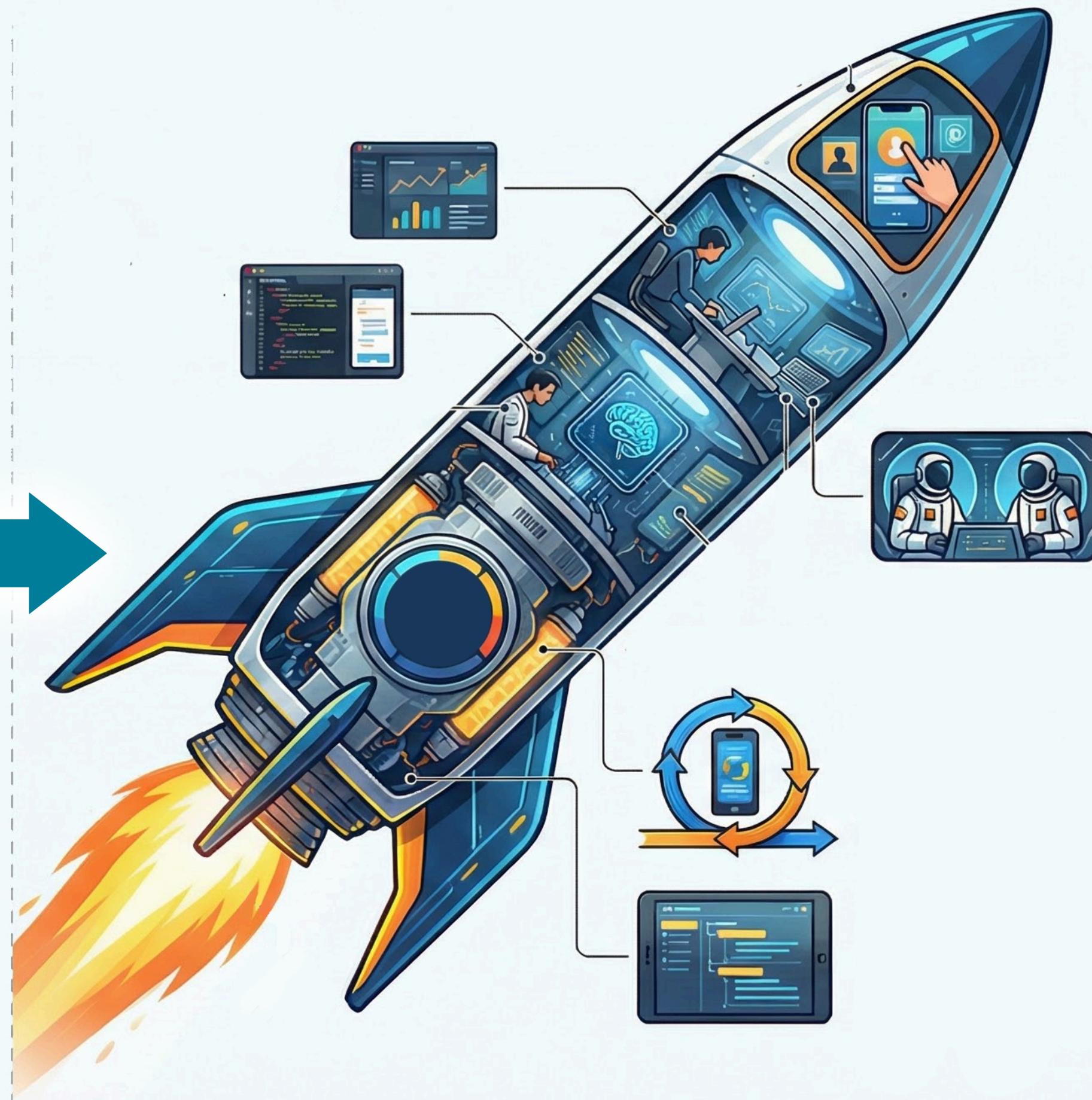
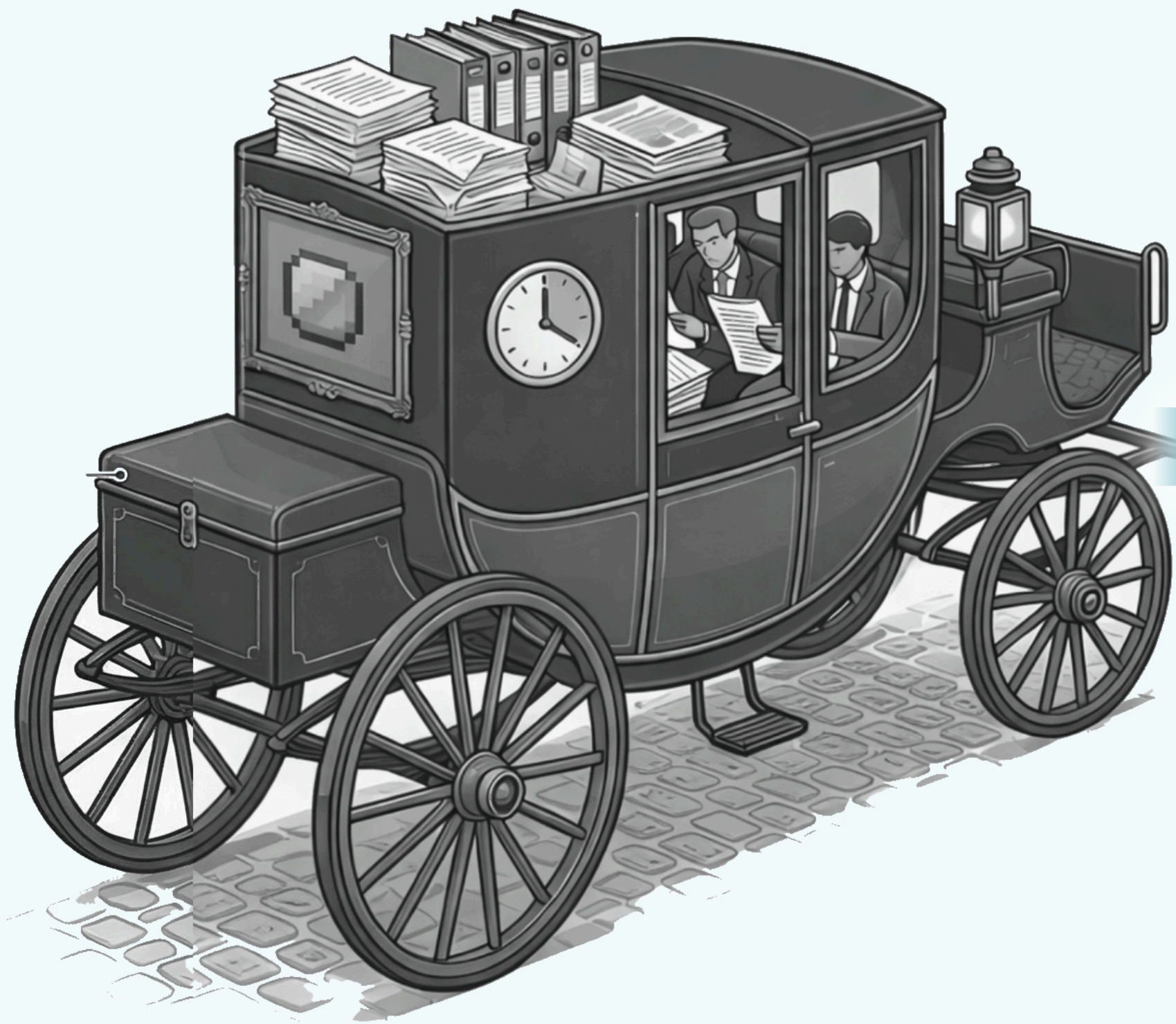








Pods Eliminate the Translation Layers That Slow Delivery

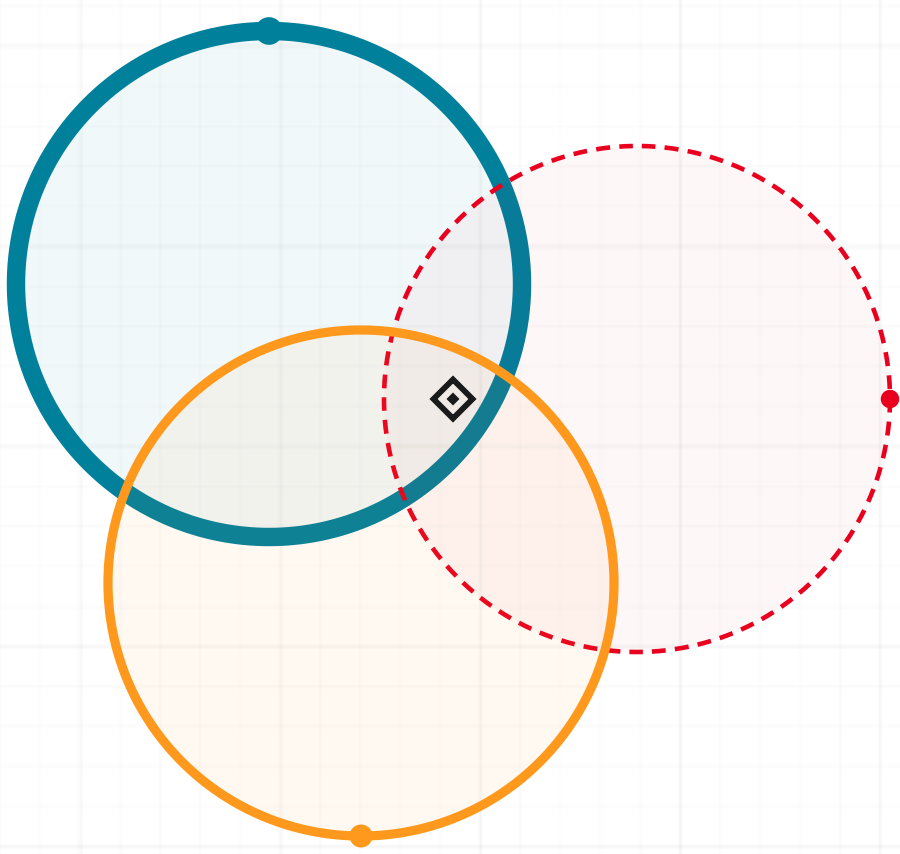
THE NEW PARADIGM

- 01 **Iterate first. Spec later.**
- 02 **Roles collapse. Handoffs disappear.**
- 03 **AI fills the operational gaps that previously required headcount.**



Every Delivery Cycle Strengthens the Next Iteration

Execution becomes predictable when learning never stops.



01 Discover

Synthesize stakeholder signals, system context, and competitive inputs to define a differentiated solution direction.

DELIVERABLE: SIGNED-OFF DOCUMENTATION + PROMPT PACKAGE

02 Iterate

Iterate through working Canvas-driven iterations with same-day feedback and pre-code stakeholder alignment. Then build, integrate, and continuously test against acceptance criteria.

DELIVERABLE: VALIDATED ITERATION READY FOR PRODUCTION SCAFFOLDING

03 Launch

Ship to production users, run retrospective, capture learnings, and feed evidence back into the next Discover cycle.

DELIVERABLE: RELEASE + RETRO INTELLIGENCE REPORT



Two Operators Drive Each Pod Everyone Else Advises

Execution cores, not committees. Built for speed and decisive action.

CORE OPERATORS

 Engineer

 Champion

AVAILABLE SUPPORT · ON DEMAND

UX

QA

PM

CS

Engineer and Champion drive the pod.
Everyone else advises.

AI is the force multiplier. Every person
commands unprecedented leverage.

AI Collapses Roles, Replacing Sequential Handoffs with Shared Iteration

From passing documents over weeks to iterating in code over hours.

The Old Way: Silos

- ✗ PM writes specs for weeks
- ✗ Designer lives in Figma
- ✗ Engineer builds blindly from specs

Result: Context decays across slow phase gates.

The New Way: Iteration

- ✓ Anyone can shape the product
- ✓ Champion drafts clickable UI
- ✓ Engineer orchestrates AI codebase

Result: Zero handoffs. Teams of 10 become 3.



Context Synthesis Replaces Multi-Week Spec Cycles

What used to require heavy manual synthesis now takes a morning with AI.

Gather Context

Dump transcripts, tickets, and signals straight into **AI**.

Output: Problem summary highlighting what's broken.

[ChatGPT](#) [Teams](#)

Draft Documentation

Generate a tight 1-page spec focused purely on core flows.

Output: High-fidelity scope and constraints.

[ChatGPT](#) [Copilot](#)

Define Done

Lock the exact criteria for what "shipped" means before building.

Output: Testable acceptance criteria.

[ChatGPT](#) [Copilot](#)



Designers Ship Working UI in Hours, Not Weeks

A designer with VS Code can produce a clickable artifact in a single session.

Shape the Brief

Pressure-test specs with UX thinking and find gaps using **AI**.

Output: UX-refined iteration brief.

ChatGPT

Copilot

Build in Code

Generate full, working interface views via plain language—no static mockups.

Output: Clickable, testable iteration.

VS Code

Opus 4.6

Codex

Iterate Live

Refine live by prompting AI. Act as creative director, not a pixel pusher.

Output: Rapid live adjustments.

VS Code

Opus 4.6



Engineers Orchestrate AI, Not Write Every Line

Engineers direct, integrate, and ensure quality while AI handles the heavy lifting.

Scaffold the App

Point **AI** at the codebase to scaffold production code directly from UI drafts.

Output: Initial flows running in 48hrs.

VS Code Opus 4.6 Codex

Ship and Iterate

Automate boilerplate (tests, APIs, models) to shift focus from syntax to direction.

Output: Features shipped in minutes.

VS Code Opus 4.6 Codex

Own Architecture

Map integrations and use AI as a peer reviewer to catch blind spots early.

Output: Deployable structure Day 1.

ChatGPT VS Code

**QA Is Consultative, Not Gating.
Stakeholders Clear the Path.**

QA advises and automation covers the baseline. Stakeholders remove friction, not approve output.

QA Advisory

Act as scope consultants, not embedded gatekeepers. Use **AI** to generate test scenarios.

Output: Focus on edge cases, not manual execution.

ChatGPT

VS Code

Automated Coverage

Baseline QA coverage is generated entirely by automation during the build.

Output: Tested alongside every code change.

VS Code

Opus 4.6

Stakeholder Alignment

Provide fast yes/no answers on scope. Absorb org friction so the pod can ship. No committees.

Output: Strategic direction without blockers.



Every Artifact Must Accelerate Testing or Iteration

The artifact is the iteration. Everything else is a note to make the iteration better.

What Pods Don't Produce

- ✗ Multi-page spec documents
- ✗ Polished Figma handoff files
- ✗ Status decks or approval presentations
- ✗ Documentation drafted before the software works

What Pods Produce

- ✓ **Rapid Project Plan (RPP)** as the baseline
- ✓ Clickable, code-backed iterations
- ✓ AI-generated test checklists and edge cases
- ✓ Short decision logs instead of meetings



[MANDATORY PROTOCOL]

The RPP Is the Required Entry Point Before Any Pod Ships

Seven fields. One page.

The foundation every pod builds against.

Problem

What problem are we solving right now?

Scope Boundary

What is explicitly out of scope?

Primary User

Who exactly runs the workflow?

Test Workflow

Exact capture-to-review flow.

Definition of Done

Operational · Usable · Demonstrable

Success Signal

What proves value quickly?

Owner

Champion responsible for execution.

RPP in the Wild: The Miami Dade Fast Track

A real execution plan stripped of the noise. Focus entirely on reality and constraints.

EXECUTION CORE

Jonathan

CHAMPION

Vlado

ENGINEER

The Core Problem

Deep folder trees degrade performance for enterprise clients (~4000 folders). API workarounds are failing.

Aggressive Scope

Rip out legacy folder logic. Migrate entirely to the new domain permission model. Stabilize performance before touching UI.

Harsh Reality

The Euro engineering team owns the implementation but is severely resource constrained without external support.

Phased Execution

Phase 1 locks performance. Phase 2 introduces bulk editing UI. Phase 3 generalizes across the core enterprise.

Success

Large massive projects load instantly without any degradation.

The underlying artifact.

Miami Dade Fast Track

TL;DR

Stabilize File Manager performance for large folder structures by migrating folder permissions to the new domain permission model, then deliver bulk permission management to eliminate folder-by-folder edits.

Success = large projects load without degradation, admins modify permissions across thousands of folders efficiently.

What We're Shipping

- **Backend Architecture:** Migrate to new domain permission model
- **Performance Stabilization:** Eliminate deep folder permission crawl
- **UI Enhancement:** Bulk permission management workflow

Problem Breakdown

Performance

- Deep folder trees degrade performance
- Thousands of folders per project
- API workarounds do not scale

Usability

- Manual edits per folder required
- Miami Dade: ~4000 folders, ~75 groups
- Current UI not viable at enterprise scale

Goal

Ensure File Manager supports enterprise-scale folder structures without performance degradation while enabling efficient bulk permission management.

Scope — Phase 1: Performance

- Replace folder permission logic with new domain model
- Align folder architecture with partition-level permissions
- Validate performance on large data sets

Out of scope: usability enhancements before performance stabilization

Scope — Phase 2: Usability

- Introduce bulk permission editing workflow
- Enable cross-folder updates in a single action
- Ensure solution benefits all customers, not just Miami Dade

Core Technical Decisions

- Adopt onboarding rewrite permission model for folders
- Avoid API-based mass permission mutation
- Performance validation before UI improvements begin

Constraints & Trade-offs

- Euro engineering team owns implementation
- Euro team is resource constrained
- Performance must be solved before usability
- High-visibility customers involved
- No external engineering support for Euro team

Phased Plan

Phase 1: Define migration approach · Assign engineering ownership · Validate performance on large projects

Phase 2: Design bulk permission UX · Implement efficient update workflow · Test across Miami Dade and GSA

Phase 3: Generalize rollout across enterprise · Monitor performance and feedback

Stakeholders

- Engineering Architecture: Colin
- Implementation: Vlad and Euro Team
- API Context: Mike Richie
- Customer Success: Kimberly Day
- Partner Enablement: John Quigley

Shared Vocabulary: Eliminating Misalignment

Consistent terms prevent drift.

Every pod behaves the exact same way.

Pod

01

A small execution unit. Not a traditional team. A focused delivery core designed to move at the speed of decisions.

Champion

02

The single person closest to the problem who drives scope, removes blockers, and makes decisions. Accountable for what ships.

Advisor

03

Expertise (UX, PM, QA, Platform) contributing exactly when the pod needs it. Supports progress rather than gating it.

Rapid Project Plan

04

A short execution source of truth. Defines the problem, scope boundary, and definition of done. Extremely fluid.

Inputs

05

Raw context (transcripts, tickets, interviews) fed directly into AI. This permanently replaces sequential hand-off deliverables.

Iteration

06

A working representation of the workflow. The entire point is to get something in front of people fast to gather feedback.

Iteration Loop

07

Discover, Iterate, Validate, Ship, Repeat. The tempo explicitly adapts to the problem. Each pass rapidly accelerates.

Done

08

Operational and usable. A real user can successfully complete the core workflow. Does not mean fully polished or documented.

Demo

09

A working demonstration inside the real system. If it can't be shown genuinely working, it simply isn't ready.



Continuous Progress Beats Delayed Polish

Speed doesn't come from working harder. It comes from closing the feedback loop instantly.

Visible Progress

The working iteration advances continuously.

System-Level Feedback

Stakeholder reaction happens entirely inside the real system, not in abstract review meetings.

Cross-Pod Diffusion

Successful patterns spread immediately across all execution teams.

Execution is the strategy. The pod that ships fastest teaches the org the most.



Done Means Operational.
~~Not Polished.~~

What "Done" Means

- Operational**
The workflow executes end to end.
- Testable**
A real human can actively manipulate it.
- Adoptable**
A user can complete the primary job-to-be-done.
- Demonstrable**
It can be shown live in the system, not on slides.

What It Does Not Mean

- ~~**Pixel Perfect**~~
Visual polish is secondary to functional proof.
- ~~**Fully Documented**~~
The working system serves as the source of truth.
- ~~**All Edge Cases Handled**~~
We solve the core flow before hunting exceptions.



**Teams no longer operate
around a PRD. They operate
around an Iteration.**

CASE STUDY

kCapture Proves the Model

From an abstract concept to a shipping product without a single traditional handoff.

EXECUTION CORE

ProCon

CHAMPION

Colin

ENGINEER

ALREADY SHIPPED

- | Floor plans with persistent pins
- | Capture timelines (photo, video, 360)
- | Live mobile 360 panorama
- | Offline-tolerant sync queue

WHAT IT PROVES

- | Zero-training capture workflows
- | Flawless automated pin binding
- | Clear location-history traversal

Spatial inspection over media storage. Capture attaches to place, not gallery.

[illegible]



Canvas Validated the Execution Model

The pilot pod. Canvas shipped as a full product using the execution framework before we even formalized it.

EXECUTION CORE

Pritesh CHAMPION

Colin ENGINEER

WHAT IT PROVED

- | Working product shipped in weeks, not months.
- | AI accelerated every phase of the development lifecycle.
- | Stakeholder feedback evaluated completely within live iterations.
- | Spec-first pipeline fully abandoned for: problem → iterate → ship.

Canvas is the proven pattern. Subsequent pods scale it into standard.



**One execution model.
Three proving grounds.**

01

VALIDATION SEQUENCE



Calendar Pod

EXECUTION CORE

Steve CHAMPION

Colin ENGINEER

Denny, Bridgette ADVISORS

- | Proving surface for rapid execution on live workflows.
- | Establishes pattern for Submittals, Inspections, and Closeout.
- | The primary proving ground. Anchors org-wide expansion.

02

VALIDATION SEQUENCE



Strategic Pod

EXECUTION CORE

Sakiba, Jonathan CHAMPIONS

Vlado ENGINEER

Brian Haines ADVISOR

- | Strategic execution on AssetCentric PM expansion surfaces.
- | Leverages existing app docs as fuel for immediate builds.
- | Validates framework for core platform evolution.

03

VALIDATION SEQUENCE



Forward Pod

EXECUTION CORE

- Jordan CHAMPION
- TBD ENGINEER
- TBD ADVISORS

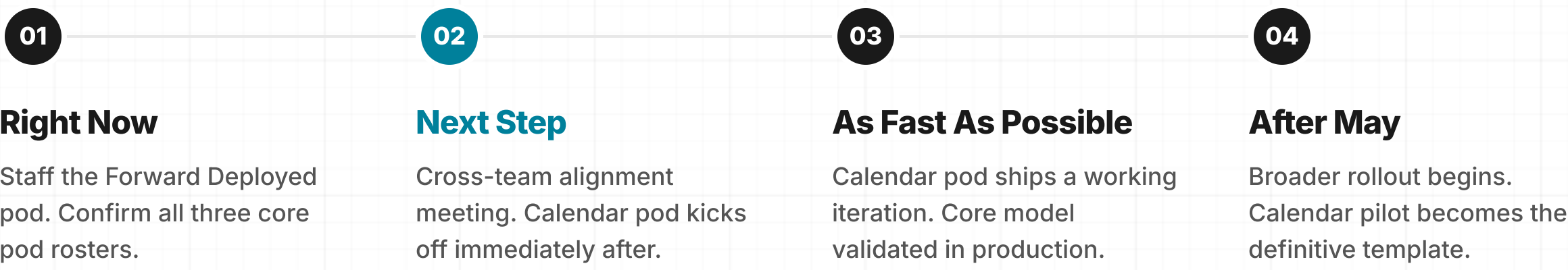
- | Embeds directly with customers for live ground-truth execution.
- | Real-time workflow discovery paired with instant implementation.
- | Pending dedicated engineering support before full activation.



● **TIMELINE & ROLLOUT**

Calendar pod is the pilot.

Model expands post-May.



■ EXECUTION PROTOCOL

Six Concrete Behavior Shifts

These are not aspirational. These start now.

01

Ship To Learn
No more weeks of polish before learning. Ship unfinished but fully operational iterations.

02

Same-Day Artifacts
PMs ship a working iteration the same day they write documentation. Problem frames become live code.

03

Designers in Code
Designers move into VS Code. Final layout ships as working HTML structures, not static Figma frames.

04

Engineers Direct AI
Engineers direct AI logic instead of writing every line. Scaffolding, iteration, and testing are accelerated.

05

Embedded QA
QA is everyone's job, organically embedded from day one. There is no rigid downstream handoff.

06

Centralized Artifacts
Pod execution pacing, coordination, and live prototype tracking are owned and monitored centrally.

APPENDIX

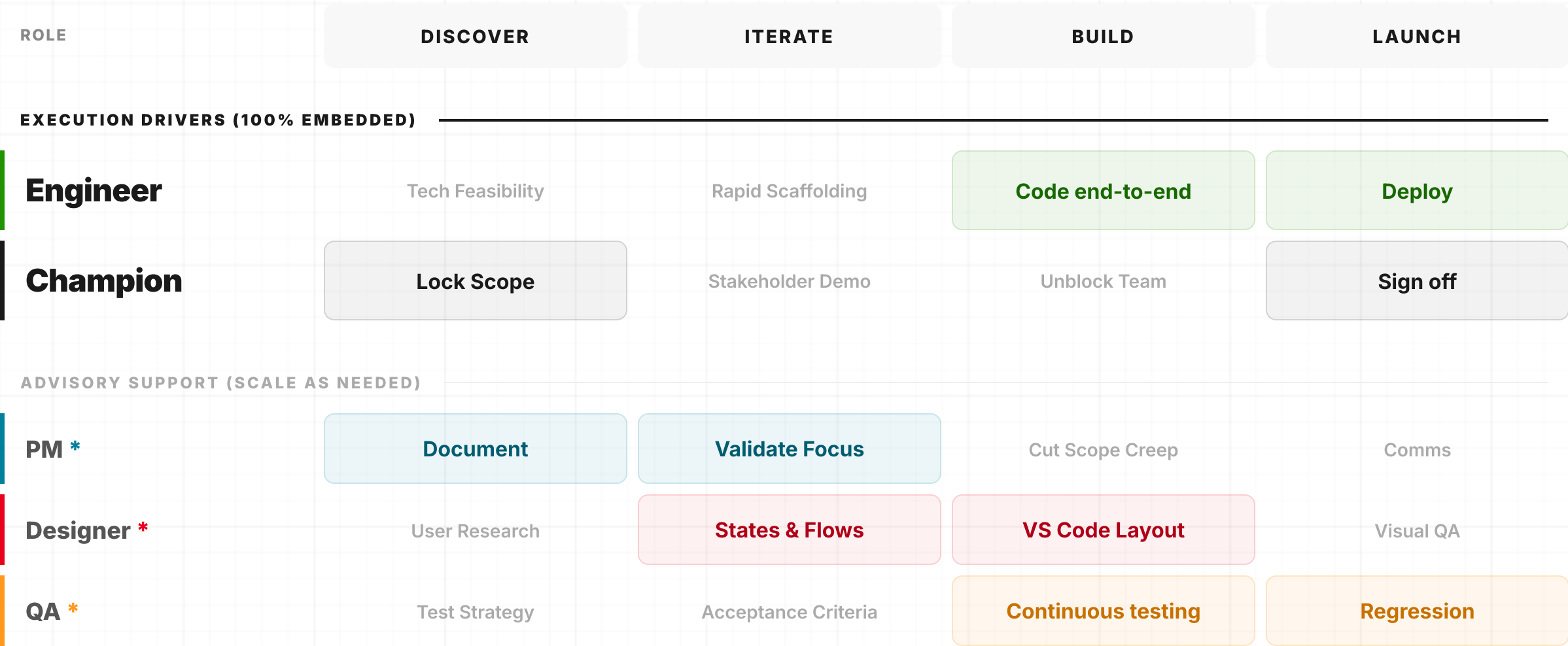
Pod Execution System

25

Engineer & Champion

Are Constant. All Others Flex.

No downstream handoffs. The entire pod is active in every phase. Highlighted cells indicate **execution drivers**, while faint cells represent active cross-functional support.



* **Advisory Functions:** Can be filled by the Engineer, Champion, or an on-demand specialist depending on pod composition. Not required headcount.

26

Single-Customer Insights Become Platform Capabilities.

Pods execute highly specific, local workflows. When local patterns overlap across multiple pods, that validated insight graduates back into the core platform for everyone.

01 THE POD LEVEL

Local R&D

A single Pod rapidly builds, tests, and validates a hyper-specific workflow directly with one real customer.

02 THE NETWORK LEVEL

Market Signal

Multiple pods independently detect the exact same workflow pattern and need emerging across different customers.

03 THE GLOBAL LEVEL

Platform Scale

The proven workflow is extracted, standardized, and permanently deployed as a core Kahua platform capability.

THE OLD WAY

~~Spec~~ Author

- ✗ Spends weeks writing detailed PRDs
- ✗ Delays engineering for requirements
- ✗ Measures success by document volume

THE NEW WAY

Decision Scientist

The PM's absolute value is judgment and velocity. Dictate the flow, synthesize the context, and rely on real validation instead of static specifications.

Live Iteration

Define problem frames entirely inside working UI models on day one.

Instant Synthesis

Pipe all customer signals directly into Canvas to synthesize architecture immediately.

Real Validation

Force users to interact with live workflows instead of reviewing static ideas.

THE OLD WAY

PixelPusher

- ✗ Spends weeks drawing static mockups
- ✗ Delivers untested redline annotations
- ✗ Forces disconnected handoff cycles

THE NEW WAY

Creative Director

Design accelerates iteration instead of preceding it. You direct output logic and shape fully interactive features, eradicating static files entirely.

Live Prototyping

Open the codebase directly and command AI to scaffold interactive UI.

Output Curation

Direct the generated logic flow instead of manually aligning pixels.

Native Handoff

Abandon Figma links and ship functional logic natively alongside engineers.

THE OLD WAY

ManualCoder

- ✗ Waits for visual mockups to begin
- ✗ Spends months typing boilerplate
- ✗ Gates code review until phase completion

THE NEW WAY

AI Orchestrator

The engineer's value is system orchestration and supreme judgment, not physical keystrokes.

Deep Scaffolding

Generate the core architectural infrastructure instantly using massive context prompts.

Total Ownership

Assume immediate control of environments, component architecture, and CI integrations.

Rapid Velocity

Deliver end-to-end, functional user flows in staging within a 48 hour span.

THE OLD WAY

Gatekeeper

- ✗ Waits for engineering to declare "done"
- ✗ Follows manual, fragile regression plans
- ✗ Acts as an adversarial release gate

THE NEW WAY

Embedded Partner

Validation is continuous and automated natively. There is no concept of an adversarial "handoff to QA" at the finish line.

Day One Entry

Merge into the iteration cycle at phase zero to establish safety bumpers.

Multi-Agent Scenarios

Orchestrate autonomous AI agents capable of relentlessly blitzing edge cases.

Velocity Enablement

The core objective shifts from artificially blocking builds to massively accelerating deployment.

HORSE AND CARRIAGE

The Old Way (The Carriage)

DETAILED PRDs
From Sequential Specs

STATIC MOCKUPS

MANUAL LINE-BY-LINE CODE
Pixel Perfection

Legacy Goal:
Pixel Perfection

Product Manager
Writes weeks of detailed PRDs

Designer
Static Figma mockups/annotations

Engineer
Manual line-by-line coding

Supporting Detail:
The old way loses weeks in translation

SLOW, SEQUENTIAL HANDOFFS

ROCKET SHIP

The New Way (The Rocket)

DEFINITION: The New "Definition of Done"
Operational Utility Over Pixel Perfection
"Done" means a real user can complete a workflow, not that it is polished.

Product Manager
Operates as a Decision Scientist

Designer
Ships working HTML via VS Code

Engineer
Orchestrates AI and Architecture

The Core Strike Team



Two Operators Drive the Mission
An Engineer and a "Champion" (Decision Maker) lead, everyone else advises on-demand

AI-AUGMENTED OUTPUT
Iteration-Centric



PROCESS_FLOW: The 48-Hour Loop
Rapid Workflow Validation
Primary user flows must run end-to-end in the real app within 48 hours

KEY_FINDING: The Rapid Project Plan
The Living Entry Artifact
A single source of truth defining scope, problem, and "Definition of Done."

